



Puffer Preconf & Puffer Institutional Security Review

Cantina Managed review by:

Gerard Persoon, Lead Security Researcher

Ladboy233, Security Researcher

August 12, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	startRestakingValidators() doesn't interact with eigenlayer	4
3.1.2	getMerkleRoot returns empty merkle root when pendingMerkleRoot is not set (e.g. after cancelPendingMerkleRoot)	4
3.2	Low Risk	4
3.2.1	setNewMerkleRoot() with same newMerkleRoot	4
3.2.2	Not all init functions are called	5
3.2.3	Denial of service in withdrawNonRestakedETH()	5
3.2.4	balanceBefore and balanceAfter is vulnerable with reentrancy attacks	5
3.3	Gas Optimization	6
3.3.1	Variables could be cached	6
3.3.2	Function registerClaimer() can be optimized	6
3.3.3	Function _startValidators() can be optimized	6
3.4	Informational	7
3.4.1	UnifiRewardsDistributor could use ReentrancyGuardTransient	7
3.4.2	No explicit error message in _processClaims()	7
3.4.3	Unused EnumerableSet.UintSet	7
3.4.4	abi.encodeCall() can be used	8
3.4.5	avsOperatorStatus() is deprecated	8
3.4.6	Unnecessary typecasts	8
3.4.7	Validators can't be registered again	9
3.4.8	Creation of NonRestakingWithdrawalCredentials can be simplified	9
3.4.9	ERC4626 compatibility	9
3.4.10	__deprecated_withdrawer doesn't require a value	9
3.4.11	completeQueuedWithdrawals() and token address 0	10
3.4.12	completeQueuedWithdrawals() behaves differently than withdrawNonRestakedETH()	10
3.4.13	requestEigenPodConsolidation() doesn't check array lengths	10
3.4.14	Incorrect comment in function requestEigenPodConsolidation()	10
3.4.15	Incorrect comment in getWithdrawalCredentials()	11
3.4.16	Commit code is different than onchain code	11
3.4.17	Duplicate structs	12
3.4.18	Some functions can't be called via InstitutionalVault	12
3.4.19	Collisions with concatenation of dynamic types	12
3.4.20	Additional emit in EigenPod::requestWithdrawal()	12
3.4.21	No return of excess ETH	13
3.4.22	Unused errors	13
3.4.23	memory in function parameters	13
3.4.24	Incorrect comment in createVault()	14
3.4.25	Consider validating if the user is authorized to make custom delegatecall	14

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	<i>Must fix as soon as possible (if already deployed).</i>
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Puffer is a decentralized native liquid restaking protocol (nLRP) built on Eigenlayer. It makes native restaking on Eigenlayer more accessible, allowing anyone to run an Ethereum Proof of Stake (PoS) validator while supercharging their rewards.

From Jul 14th to Jul 16th the Cantina team conducted a review of [puffer-institutional](#) and [Puffer-Preconf](#), on PR 15 and commit hash 4346c762 respectively. The team identified a total of **34** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	4	4	0
Gas Optimizations	3	3	0
Informational	25	14	11
Total	34	23	11

3 Findings

3.1 Medium Risk

3.1.1 `startRestakingValidators()` doesn't interact with eigenlayer

Severity: Medium Risk

Context: [InstitutionalVault.sol#L247-L254](#), [InstitutionalVault.sol#L273](#), [InstitutionalVault.sol#L293-L302](#)

Description: `startRestakingValidators()` does give the right to withdraw to eigenlayer. This means eigenlayer can't use it to restake (fully understood with help of the project). Also an additional `cli` action is required to register with eigenlayer. This is not clear in the documentation.

Recommendation: Investigate what should be done to give the right to withdraw to eigenlayer. Consider making the required actions clear in the documentation. Also consider adding a comment at function `startRestakingValidators()` making clear additional actions are required.

Puffer Finance: Added to the documentation in [puffer-institutional PR 31](#).

Cantina Managed: Fix verified.

3.1.2 `getMerkleRoot` returns empty merkle root when `pendingMerkleRoot` is not set (e.g. after `cancelPendingMerkleRoot`)

Severity: Medium Risk

Context: [UnifiRewardsDistributor.sol#L116-L121](#), [UnifiRewardsDistributor.sol#L147-L156](#), [UnifiRewardsDistributor.sol#L203-L209](#)

Description: When a user claims the token reward, the code calls `getMerkleRoot()` to retrieve the merkle root. When `pendingMerkleRoot` is not set then `block.timestamp >= pendingMerkleRootActivationTimestamp` is always true and then an empty merkle root is returned. This blocks the user from claiming the reward.

This situation happens when `cancelPendingMerkleRoot()` is called: both `pendingMerkleRoot` and `pendingMerkleRootActivationTimestamp` are reset to 0.

After the `pendingMerkleRoot` is set, the user needs to wait for 7 days to make `block.timestamp >= pendingMerkleRootActivationTimestamp` return true. Repeatedly doing `cancelPendingMerkleRoot()` can result in a complete denial of service for claiming (this normally shouldn't happen).

Recommendation: The code has to validate if `pendingMerkleRoot` is empty as well when retrieving `merkleRoot`:

```
function getMerkleRoot() public view returns (bytes32) {
-   if (block.timestamp >= pendingMerkleRootActivationTimestamp) {
+   if (pendingMerkleRoot != bytes32(0) && block.timestamp >= pendingMerkleRootActivationTimestamp) {
        return pendingMerkleRoot;
    }
    return merkleRoot;
}
```

Puffer Finance: Fixed in [Puffer-Preconf PR 53](#).

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 `setNewMerkleRoot()` with same `newMerkleRoot`

Severity: Low Risk

Context: [UnifiRewardsDistributor.sol#L132-L141](#)

Description: When `setNewMerkleRoot()` is called with the same `newMerkleRoot`, then the `pendingMerkleRootActivationTimestamp` is shifted forward, which leads to a delay in activation (assuming it is done before the `pendingMerkleRootActivationTimestamp`). If accidentally (due to a configuration error), this is done repeatedly, the `newMerkleRoot` will never be activated.

Recommendation: Consider checking that `newMerkleRoot!=pendingMerkleRoot`.

Puffer Finance: Fixed in [Puffer-Preconf PR 54](#).

Cantina Managed: Fix verified.

3.2.2 Not all `init` functions are called

Severity: Low Risk

Context: [InstitutionalVault.sol#L79-L87](#), [UniFiAVSManager.sol#L115-L116](#)

Description: Not all `init` functions are called in the `initialize()` functions. The risks is limited because these functions are currently empty. However that could change in the future.

Recommendation: Consider also calling the following:

- In `initialize()` of `UniFiAVSManager`:
 - `__UUPSUpgradeable_init()`.
 - `__Context_init()`.
- In `initialize()` of `InstitutionalVault`:
 - `__Context_init()`.

Puffer Finance: Fixed in [puffer-institutional PR 28](#) and [Puffer-Preconf PR 54](#)

Cantina Managed: Fix verified.

3.2.3 Denial of service in `withdrawNonRestakedETH()`

Severity: Low Risk

Context: [InstitutionalVault.sol#L399-L409](#), [InstitutionalVault.sol#L401-L409](#), [NonRestakingWithdrawalCredentials.sol#L61](#)

Description: When redrawing the last ETH from `NonRestakingWithdrawalCredentials` with `withdrawETH()`, the following calculation will revert if some additional ETH has been donated to the `NonRestakingWithdrawalCredentials` contract:

```
$.nonRestakedValidatorsETH -= uint128(difference);
```

This could lead to a denial of service of the call to `withdrawNonRestakedETH()`. This can be fixed with `setValidatorsETH()`, but in the mean time additional ETH can be donated.

Note: a donation of 1 Wei is sufficient.

Recommendation: Consider not reverting when `$.nonRestakedValidatorsETH < uint128(difference)`. This can be done with `saturatingSub()`

Puffer Finance: Fixed in [puffer-institutional PR 30](#).

Cantina Managed: Fix verified.

3.2.4 `balanceBefore` and `balanceAfter` is vulnerable with reentrancy attacks

Severity: Low Risk

Context: [InstitutionalVault.sol#L401-L409](#)

Description: The construction of using `balanceBefore` and `balanceAfter` is vulnerable with reentrancy attacks. Because in that situation the same `balanceAfter` could be counted for multiple reentrant calls. This is unlikely to happen because:

- `withdrawNonRestakedETH()` has access control via `restricted`.
- `noRestakingWithdrawalCredentials` points to a known contract.

However error is authorizations in combination with `customDelegateCall()` could change this.

Recommendation: To eliminate the risk, consider adding a `nonReentrant` modifier.

Puffer Finance: Due to refactoring `customDelegateCall()` was removed in [puffer-institutional PR 20](#).

Cantina Managed: Fix verified.

3.3 Gas Optimization

3.3.1 Variables could be cached

Severity: Gas Optimization

Context: [InstitutionalVault.sol#L287](#), [InstitutionalVault.sol#L293](#), [InstitutionalVault.sol#L295](#), [InstitutionalVault.sol#L298](#), [InstitutionalVault.sol#L385](#), [UniFiAVSManager.sol#L177](#), [UniFiAVSManager.sol#L222](#), [UnifiRewardsDistributor.sol#L105](#), [UnifiRewardsDistributor.sol#L107](#), [UnifiRewardsDistributor.sol#L110](#), [UnifiRewardsDistributor.sol#L117](#), [UnifiRewardsDistributor.sol#L165](#), [UnifiRewardsDistributor.sol#L166](#)

Description: Several variables could be cached so save gas, provided there is no stack issue.

Recommendation: Consider caching the following:

- Array lengths in for loops.
- Indexed variables in a for loops, that are used multiple times.
- Call to functions inside a for loop, that always give the same result, like `getMerkleRoot()` and `getWithdrawalCredentials()`.

Puffer Finance: Some optimizations were applied in [Puffer-Preconf PR 54](#), while others were left as is, because they caused stack too deep errors.

Cantina Managed: Fix verified.

3.3.2 Function `registerClaimer()` can be optimized

Severity: Gas Optimization

Context: [UnifiRewardsDistributor.sol#L164-L178](#)

Description: Function `registerClaimer` can be optimized by taking actions outside the for loop.

Recommendation: Consider changing the code in the following way:

```
function registerClaimer(address claimer, PubkeyRegistrationParams[] calldata params) external {
+   BLS.G1Point[] memory g1Points = new BLS.G1Point[](2);
+   BLS.G2Point[] memory g2Points = new BLS.G2Point[](2);
+   g1Points[0] = NEGATED_G1_GENERATOR();
    for (uint256 i = 0; i < params.length; ++i) {
        // ...
-       BLS.G1Point[] memory g1Points = new BLS.G1Point[](2);
-       g1Points[0] = NEGATED_G1_GENERATOR();
        g1Points[1] = ...
        // ...
-       BLS.G2Point[] memory g2Points = new BLS.G2Point[](2);
        g2Points[0] = ...
        g2Points[1] = ...
        // ...
    }
}
```

Puffer Finance: Fixed in [Puffer-Preconf PR 54](#).

Cantina Managed: Fix verified.

3.3.3 Function `_startValidators()` can be optimized

Severity: Gas Optimization

Context: [InstitutionalVault.sol#L273](#), [InstitutionalVault.sol#L286-L289](#)

Description: In function `_startValidators()`, the conversion with `_convertGweiToWei()` could be after the loop to save some gas.

Recommendation: Consider changing the code in the following way:

```
uint256 totalAmount;
for (uint256 i = 0; i < pubKeys.length; i++) {
-   totalAmount += _convertGweiToWei(amountsInGwei[i]);
+   totalAmount += amountsInGwei[i];
}
+   totalAmount = _convertGweiToWei(totalAmount);
```

Puffer Finance: Fixed in [puffer-institutional PR 26](#).

Cantina Managed: Fix verified.

3.4 Informational

3.4.1 UnifiRewardsDistributor **could use** `ReentrancyGuardTransient`

Severity: Informational

Context: [UnifiRewardsDistributor.sol#L6](#)

Description: Contract `UnifiRewardsDistributor` uses `ReentrancyGuard`. However there is also a more gas efficient version available, using transient storage.

Recommendation: Consider using `ReentrancyGuardTransient`

Puffer Finance: Acknowledged.

Cantina Managed: Acknowledged.

3.4.2 No explicit error message in `_processClaims()`

Severity: Informational

Context: [UnifiRewardsDistributor.sol#L96](#), [UnifiRewardsDistributor.sol#L110-L111](#)

Description: The assignment `uint256 amountToClaim = amounts[i] - claimedSoFar`; reverts when `amounts[i] < claimedSoFar`, without giving an explicit error message.

Recommendation: Consider giving an explicit error message when `amounts[i] < claimedSoFar`. One solution would be to use `saturatingSub()` in combination with the check for `amountToClaim == 0`.

Puffer Finance: Acknowledged.

Cantina Managed: Acknowledged.

3.4.3 Unused `EnumerableSet.UintSet`

Severity: Informational

Context: [UniFiAVSManager.sol#L20-L22](#)

Description: In contract `UniFiAVSManager`, `EnumerableSet.UintSet` isn't used.

Recommendation: Consider removing this line:

```
- using EnumerableSet for EnumerableSet.UintSet;
```

Puffer Finance: Fixed in [Puffer-Preconf PR 54](#).

Cantina Managed: Fix verified.

3.4.4 `abi.encodeCall()` can be used

Severity: Informational

Context: [UniFiAVSManager.sol#L65-L72](#)

Description: Function `_getAvsOperatorStatus()` uses `abi.encodeWithSelector()`, which doesn't do checks so errors are not detected.

Recommendation: Consider using `abi.encodeCall()`:

```
import { IAVSDirectoryExtended } from "../interfaces/EigenLayer/IAVSDirectoryExtended.sol";`
// ...
-     abi.encodeWithSelector(bytes4(keccak256("avsOperatorStatus(address,address)")), address(this), operator)
+     abi.encodeCall(avsOperatorStatus, (address(this), operator));
// ...
```

Puffer Finance: The reason we use `encodeWithSelector` is because `avsOperatorStatus()` is not defined (in the interface) and we don't want to include the entirety of the AVS Directory interface.

Cantina Managed: Acknowledged.

3.4.5 `avsOperatorStatus()` is deprecated

Severity: Informational

Context: [UniFiAVSManager.sol#L65-L72](#)

Description: According to a comment in [AVSDirectoryStorage.sol](#), `avsOperatorStatus()` is deprecated:

```
/// @notice Returns the registration status of each `operator` for a given `avs`.
/// @dev This storage will be deprecated once M2-based deregistration is removed.
mapping(address avs => mapping(address operator => OperatorAVSRegistrationStatus)) public avsOperatorStatus;
```

Recommendation: Consider changing the code to be compatible with the situation where `avsOperatorStatus()` can no longer be called.

Puffer Finance: Updated the AVS in [PR 56](#) to implement new methods of the upcoming EigenLayer Slashing upgrade as AVSDirectory is being deprecated. Changes:

- Removed 2-step operator deregistration because operator deregistrations are now processed by EigenLayer regardless of the callback reverts or not.
- Uses AllocationManager instead of AVSDirectory
- Implements OperatorSets required by the new EigenLayer slashing update.

Cantina Managed: Fix verified.

3.4.6 Unnecessary typecasts

Severity: Informational

Context: [UniFiAVSManager.sol#L90-L94](#), [UniFiAVSManager.sol#L110-L111](#)

Description: Several typecasts in the constructor of `UniFiAVSManager` are unnecessary.

Recommendation: Consider changing the code to:

```
- AVS_DIRECTORY = IAVSDirectory(address(avsDirectoryAddress));
+ AVS_DIRECTORY = avsDirectoryAddress;
- EIGEN_REWARDS_COORDINATOR = IRewardsCoordinator(address(rewardsCoordinatorAddress));
+ EIGEN_REWARDS_COORDINATOR = rewardsCoordinatorAddress;
```

Puffer Finance: Fixed in [Puffer-Preconf PR 54](#).

Cantina Managed: Fix verified.

3.4.7 Validators can't be registered again

Severity: Informational

Context: [UniFiAVSManager.sol#L158](#), [UniFiAVSManager.sol#L183-L185](#), [UniFiAVSManager.sol#L216-L235](#)

Description: Function `deregisterValidators()` keeps the `validator.index`, which means `registerValidators()` can't be done again for the same `validator / validatorPubkey`, because it requires the `validator.index` to be 0.

Recommendation: Check if there is a usecase for using the same validator again. If so, change the code to allow this. Otherwise add a comment to make this clear for future maintainers/readers.

Puffer Finance: Fixed in [Puffer-Preconf PR 53](#). Registering again is not allowed because that would allow an operator to increase its fees.

Cantina Managed: Fix verified.

3.4.8 Creation of `NonRestakingWithdrawalCredentials` can be simplified

Severity: Informational

Context: [InstitutionalVault.sol#L442-L446](#), [NonRestakingWithdrawalCredentialsFactory.sol#L28](#)

Description: The creation of the `NonRestakingWithdrawalCredentials` contract is done via `customDelegateCall()` and a factory, from `InstitutionalVault` and only has to be done once. This is a relatively complicated way. Additionally the `customDelegateCall()` is very powerful and thus increased the risks.

Recommendation: Consider creating an `initializeV2()` function with `reinitializer(2)` to do the equivalent of `deployNewStakingWithdrawalCredentials()`. Make sure to call this in one transaction with the contract upgrade to prevent frontrunning. Also consider removing `customDelegateCall()`.

Puffer Finance: Fixed in [puffer-institutional PR 20](#).

Cantina Managed: Fix verified.

3.4.9 ERC4626 compatibility

Severity: Informational

Context: [InstitutionalVault.sol#L34](#)

Description: `InstitutionalVault` doesn't fully comply to [eip-4626](#)

Recommendation: Consider doing the following:

- Return 0 in the functions `maxDeposit()` / `maxMint()` / `maxWithdraw()` / `maxRedeem()` when the related functions are paused;
- In functions `maxWithdraw()` / `maxRedeem()` take into account the maximum available ETH that can currently be retrieved: e.g. should be maximized to `WETH.balanceOf(address(this)) + (address(this).balance` and converted to shares for `maxRedeem()`.

Puffer Finance: Acknowledged. We are aware of this. The readme in the institutional repository says:

The vault is not 100% ERC4626 compliant to the interface as it accounts for assets that are not in the Vault, ↪ but it is not a problem for the current use case.

Cantina Managed: Acknowledged.

3.4.10 `__deprecated_withdrawer` doesn't require a value

Severity: Informational

Context: [InstitutionalVault.sol#L321-L335](#), [InstitutionalVault.sol#L334](#)

Description: In function `InstitutionalVault()`, `__deprecated_withdrawer` is assigned a value, however this isn't used. Therefore, a value of 0 might be used as well. This could be more gas efficient and also makes clear the value isn't used.

Recommendation: Consider changing the code to:

```
- __deprecated_withdrawer: address(this)
+ __deprecated_withdrawer: address(0)
```

Puffer Finance: Fixed in [puffer-institutional PR 26](#).

Cantina Managed: Fix verified.

3.4.11 `completeQueuedWithdrawals()` and token address 0

Severity: Informational

Context: [InstitutionalVault.sol#L321-L329](#), [InstitutionalVault.sol#L345-L352](#)

Description: Function `completeQueuedWithdrawals()` uses token address 0. This is fine if this token address isn't used, which is the case when `strategy != _BEACON_CHAIN_STRATEGY`. The related function `queueWithdrawals()` does indeed use `_BEACON_CHAIN_STRATEGY`, however `completeQueuedWithdrawals()` doesn't check/enforce this is the case. If the wrong strategy is used, a revert will occur that might be difficult to trace.

Recommendation: Consider checking/enforcing that the only strategy used in `completeQueuedWithdrawals()` is `_BEACON_CHAIN_STRATEGY`.

Puffer Finance: Fixed in [puffer-institutional PR 27](#).

Cantina Managed: Fix verified.

3.4.12 `completeQueuedWithdrawals()` behaves differently than `withdrawNonRestakedETH()`

Severity: Informational

Context: [InstitutionalVault.sol#L345-L358](#), [InstitutionalVault.sol#L401-L409](#), [InstitutionalVault.sol#L457-L467](#)

Description: The function `completeQueuedWithdrawals()` and `withdrawNonRestakedETH()` are similar but behave differently in updating the accounting. `withdrawNonRestakedETH()` decreases `nonRestakedValidatorsETH`, however `completeQueuedWithdrawals()` doesn't decrease `restakedValidatorsETH`. This can be corrected with `setValidatorsETH()`, but it would be more consistent to also do this in `completeQueuedWithdrawals()`.

Recommendation: Consider decreasing `restakedValidatorsETH` in `completeQueuedWithdrawals()`, when `receiveAsTokens[] == true`.

Puffer Finance: Fixed in [puffer-institutional PR 27](#).

Cantina Managed: Fix verified.

3.4.13 `requestEigenPodConsolidation()` doesn't check array lengths

Severity: Informational

Context: [InstitutionalVault.sol#L378](#)

Description: Function `requestEigenPodConsolidation()` doesn't check array lengths, while most other functions do, which is inconsistent.

Recommendation: Consider checking `srcPubkeys.length == targetPubkeys.length`.

Puffer Finance: Acknowledged. The transaction will revert somewhere down the line.

Cantina Managed: Acknowledged.

3.4.14 Incorrect comment in function `requestEigenPodConsolidation()`

Severity: Informational

Context: [InstitutionalVault.sol#L378-L391](#)

Description: The function `requestEigenPodConsolidation()` has the comment:

```
// The remainder is donated to the vault and not refunded to the caller
```

However `EigenPod::requestConsolidation()` does refund the excess ETH.

Recommendation: Consider changing the comment.

Puffer Finance: The vault doesn't refund to the caller, although the EigenPod does return to the vault.

Cantina Managed: Acknowledged.

3.4.15 Incorrect comment in `getWithdrawalCredentials()`

Severity: Informational

Context: `InstitutionalVault.sol#L507-L514`

Description: In function `getWithdrawalCredentials()` the following comment is no longer valid:

```
// (this contract address)
```

Recommendation: Consider removing or changing the comment.

Puffer Finance: Fixed in `puffer-institutional` PR 24.

Cantina Managed: Fix verified.

3.4.16 Commit code is different than onchain code

Severity: Informational

Context: `UniFiAVSManagerStorage.sol#L14-L25`

Description: The commit for the review is `4346c762`. This goes from `24093199`:

```
struct UniFiAVSStorage {
    // ...
    uint64 deregistrationDelay;
    EnumerableSet.AddressSet allowlistedRestakingStrategies;
}
```

To `4346c762`:

```
struct UniFiAVSStorage {
    // ...
    uint64 deregistrationDelay;
    mapping(uint8 => uint256) _deprecated_bitmapIndexToChainId;
    mapping(uint256 => uint8) _deprecated_chainIdToBitmapIndex;
    EnumerableSet.AddressSet allowlistedRestakingStrategies;
    mapping(bytes validatorPubkey => ValidatorData validatorData) validators;
    mapping(uint256 validatorIndex => bytes validatorPubkey) validatorIndexes;
    mapping(address operator => OperatorData operatorData) operators;
}
```

Which would be tricky because two mappings are inserted in between `deregistrationDelay` and `allowlistedRestakingStrategies` and thus the storage layout would be shifted. However the `onchain version` already has these additional mappings. So when upgrading from that, it would be ok.

```
struct UniFiAVSStorage {
    // ...
    uint64 deregistrationDelay;
    mapping(uint8 => uint256) bitmapIndexToChainId;
    mapping(uint256 => uint8) chainIdToBitmapIndex;
    EnumerableSet.AddressSet allowlistedRestakingStrategies;
}
```

Recommendation: Consider adding an intermediate step in the commits to allow for traceability.

Cantina Managed: This isn't easily fixable on github because the PRs are already merged. Set to acknowledged.

3.4.17 Duplicate structs

Severity: Informational

Context: [IUniFiAVSManager.sol#L56](#), [IUniFiAVSManager.sol#L75](#), [DeprecatedOperatorData.sol#L9](#), [DeprecatedOperatorData.sol#L34](#)

Description: The structs `OperatorCommitment` and `OperatorDataExtended` are defined in both `IUniFiAVSManager` and `DeprecatedOperatorData`. This is confusing for readers of the code.

Recommendation: Consider renaming the version in `DeprecatedOperatorData` to indicate they are deprecated.

Puffer Finance: Fixed in [Puffer-Preconf PR 54](#).

Cantina Managed: Fix verified.

3.4.18 Some functions can't be called via `InstitutionalVault`

Severity: Informational

Context: [InstitutionalVault.sol#L364-L371](#), [InstitutionalVault.sol#L378-L391](#), [NonRestakingWithdrawalCredentials.sol#L67](#), [NonRestakingWithdrawalCredentials.sol#L88](#)

Description: The `eigenPod` functions `requestWithdrawal()` and `requestConsolidation()` can be called via the `InstitutionalVault`. However the equivalent functions of `NonRestakingWithdrawalCredentials`: `requestWithdrawal()` and `requestConsolidation()` have to be called directly on the `NonRestakingWithdrawalCredentials` contract. This is inconsistent.

Recommendation: Consider also adding functions in `InstitutionalVault` to call the `NonRestakingWithdrawalCredentials` functions. Also consider renaming the functions to make clear for which underlying contract they are meant because they have similar names.

Puffer Finance: Acknowledged.

Cantina Managed: Acknowledged.

3.4.19 Collisions with concatenation of dynamic types

Severity: Informational

Context: [NonRestakingWithdrawalCredentials.sol#L88-L102](#)

Description: Function `requestConsolidation()` concatenates two dynamic types with `bytes.concat()`. This allows for collisions. This could be abused to emit `ConsolidationRequested` instead of `SwitchToCompoundingWithdrawalCredentials`.

Note: this is unlikely to occur because `requestConsolidation()` is authorized.

Note: the comparable contract `[EigenPod]` does [check the length of the Pubkeys](#).

Recommendation: Consider checking the length of `request.srcPubkey` and `request.targetPubkey`.

Puffer Finance: An invalid consolidation request may still emit an event on the smart contract, but they simply wouldn't get accepted by the beacon chain. Technically. You can do a consolidation request from EOA for 2 random pubkeys (of legit length) and the tx would not revert. So from when we are dealing with contracts, you can trigger the events in the function even though nothing would really happen.

Cantina Managed: Acknowledged.

3.4.20 Additional emit in `EigenPod::requestWithdrawal()`

Severity: Informational

Context: [NonRestakingWithdrawalCredentials.sol#L67-L79](#)

Description: `NonRestakingWithdrawalCredentials::requestWithdrawal()` does emit `WithdrawalRequested`. However the comparable function `EigenPod::requestWithdrawal()` can also do an emit `ExitRequested`:

```
// Emit event depending on whether the request is a full exit or a partial withdrawal
if (request.amountGwei == 0) emit ExitRequested(pubkeyHash);
else emit WithdrawalRequested(pubkeyHash, request.amountGwei);
```

Recommendation: Consider the usefulness of emitting `ExitRequested()` and consider adding it.

Puffer Finance: Added `indexed` as an alternative solution instead of creating a new event, in [PR 22](#).

Cantina Managed: Fix verified.

3.4.21 No return of excess ETH

Severity: Informational

Context: [NonRestakingWithdrawalCredentials.sol#L67](#), [NonRestakingWithdrawalCredentials.sol#L88](#)

Description: The functions `EigenPod::requestWithdrawal()` and `EigenPod::requestConsolidation()` send back excess ETH.

However the comparable functions `NonRestakingWithdrawalCredentials::requestWithdrawal()` and `NonRestakingWithdrawalCredentials::requestConsolidation()` don't do this.

Recommendation: Consider also returning excess ETH.

Puffer Finance: Acknowledged, we are dealing with dust amounts.

Cantina Managed: Acknowledged.

3.4.22 Unused errors

Severity: Informational

Context: [IUniFiAVSManager.sol#L139-L140](#)

Description: The error `CommitmentChangeNotReady()` is no longer used.

Recommendation: Consider removing the unused error(s).

Puffer Finance: Fixed in [Puffer-Preconf PR 54](#).

Cantina Managed: Fix verified.

3.4.23 memory in function parameters

Severity: Informational

Context: [IUniFiAVSManager.sol#L265](#), [IUniFiAVSManager.sol#L272-L275](#), [IUniFiAVSManager.sol#L303](#), [IUniFiAVSManager.sol#L309](#), [UniFiAVSManager.sol#L130](#), [UniFiAVSManager.sol#L143-L146](#), [UniFiAVSManager.sol#L293](#), [UniFiAVSManager.sol#L328](#)

Description: Several functions still use `memory` in the interface definition, while the implementation uses `calldata`. This is inconsistent. Additionally `setOperatorCommitment()` and `updateAVSMetadataURI()` still use `memory` in the function parameters.

Recommendation: Consider changing `setOperatorCommitment()` and `updateAVSMetadataURI()` to use `calldata`. Consider making the interface consistent with the implementation. Also inherit from the interface to make sure they are consistent:

```
- contract UniFiAVSManager is UniFiAVSManagerStorage, ... {
+ contract UniFiAVSManager is IUniFiAVSManager, UniFiAVSManagerStorage, ... {
```

Puffer Finance: Solved in [Puffer-Preconf PR 54](#) and [PR 55](#).

Cantina Managed: Fix verified.

3.4.24 Incorrect comment in `createVault()`

Severity: Informational

Context: [InstitutionalFactory.sol#L45-L47](#), [InstitutionalFactory.sol#L79](#)

Description: A comment in `createVault()` says the return parameters are: `{vault, access manager}`, however the real return parameters are `{access manager, vault}`.

Recommendation: Consider updating the comment.

Puffer Finance: Acknowledged. The factory is already deployed in [mainnet](#)

Cantina Managed: Acknowledged.

3.4.25 Consider validating if the user is authorized to make custom delegatecall

Severity: Informational

Context: [InstitutionalVault.sol#L442-L446](#)

Description: When making `customExternalCall`, the [code](#) check if the action is prohibited:

```
(bool isAllowed,) = IAccessManager(authority()).canCall(_msgSender(), target, bytes4(data));
```

However, the restricted user can bypass the restriction via `customDelegateCall`.

Recommendation: Consider adding the validation:

```
(bool isAllowed,) = IAccessManager(authority()).canCall(_msgSender(), target, bytes4(data));
```

to `customDelegateCall` as well.

Puffer Finance: Fixed in [puffer-institutional PR 20](#).

Cantina Managed: Fix verified.